

Sudoku Solver Assignment

In line with the suggestion provided within the assignment's brief, I have constructed my AI sudoku solver using an implementation of a backtracking depth-first search algorithm with constraint propagation. This project took inspiration from both the course's unit material, and the courses' suggested complementary textbook¹; both of which are referenced at the end of this report.

To represent a sudoku state, the algorithm stores a 9 x 9 matrix of "final values", that are arbitrarily set to -1 when a particular cell is unassigned. In addition to this, a given cell contains its own array containing a list of possible values that it could take, which are updated in line with each addition to the game state and any proceeding constraint satisfactions.

As stated in the unit material textbook¹, a backtracking search *is used for a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign*². In the context of our sudoku assignment, this means that the solver takes an arbitrary partial sudoku state, chooses a variable (cell) and a value for its assignment (see later optimisations for how these choices are made) and updates the state's final values and possible values accordingly. It is at this point that the algorithm has a decision to make; if the new partial state is invalid, i.e. violates the laws of Sudoku, then it will backtrack to the previous partial state, and select a different value to assign to the chosen variable and repeat this process. However, if the new partial state is in fact valid, then the process repeats for this current state, effectively going deeper and deeper into the child states that exist from our initial parent state. With respect to a 9 x 9 sudoku grid's complexity, the additional aspect of constraint satisfaction, helps prune the number of child states for each given parent.

It is in fact through these constraints, that we can heavily increase our solver's efficiency. Simply running the backtracking depth-first search, exempt from any constraint satisfactions, the 60 test puzzles are solved in a rather disappointing duration of 1 minute and 38.4 seconds. This can be drastically improved, shortening the total computation time to a mere 0.7 seconds with our added constraints, owing to the sheer reduction in possible partial game states. These are as follows: "Naked Candidates", "Hidden Candidates", "AC-3", "Intersection Removal", "X-Wing" and "Hidden Candidate" constraints. Below I have detailed a brief explanation of each, with more detail being found through their respective links provided in the referencing to this report:

- Naked Candidates – if n cells share a total n possible assignment values, then these values can be removed from the rest of the cells within their shared unit (row or column or box)⁵.
- Hidden Candidates – if n values only appear as possible assignment values for n cells within a given unit, remove all other possible values from these n cells⁶.
- Intersection Removal – if any value can only take 2/3 possible cells in a given unit, then it can be removed from any corresponding intersecting units⁷.
- X-Wing – if there are only two possible cells for a value, in each of two different units of the same kind, and these cells lie also on two other units of the same kind, then all other cells for that value can be eliminated from the latter two units⁸.
- AC-3 – if a single pair of cells within a given state do not satisfy arc consistency (for every possible value for cell x , there is at least one possible value of its neighbour cell y that satisfies the pairs constraint/s), then that state can be 'ignored'⁹.

We effectively classify this CSP (Constraint Satisfaction Problem) as a tree-search; with the start node being the inputted partial game state, and each child node being the current state 'plus a given move added to the board'. Using the formula⁴ $n! \times d^n$ to calculate the total possible number of nodes that require searching, with n and d representing the number of variables (81) and possible values for each variable (9) respectively, we are left with approximately 10^{198} potential states to check! This evidently highlights the need to rigorously prune the search space using constraints, which is why I have chosen to include the algorithms listed above.

Further optimisation techniques, include MRV (Minimum Remaining Values) which selects the cells that can take the smallest number of possible values, and LCV (Least Constraining Value) which effectively picks the "most-forgiving" or "least-restrictive" value to add to the current game state.

Whilst the project and the solver have been a clear success, as always, they are open to further improvements that could increase efficiency to an even greater extent. Firstly, "SudokuWiki" contains a further vast array of intricate, advanced constraints that could be implemented, although it would be interesting to see the trade-off between a further reduction in possible states and the increase in computational activity. Another would be the inclusion of "back-jumping"³, potentially through the use of a "conflict-directed back-jumping" algorithm. When encountering an invalid game state, the current algorithm simply returns to its immediate predecessor, whilst back-jumping attempts to locate the assignment that caused the current invalid state, avoiding any branches that do not need revisiting. This is explored in greater detail on page 219 of the unit material's textbook¹.

References:

Algorithm:

- Back-tracking depth first search: “River Crossing” Moodle Assignment (S1, W7 – Uninformed Search) <https://moodle.bath.ac.uk/mod/assign/view.php?id=1272476>
- Partial State Class: “Eight Queens with Constraint Satisfaction” Moodle Assignment (S1, W9 – Constraint Satisfaction) <https://moodle.bath.ac.uk/mod/page/view.php?id=1275315>
- MRV: Variable and Value Ordering: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 216
- LCV: Variable and Value Ordering: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 217

Research / Definitions:

1. Unit Material Textbook: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition.
2. Backtracking Search for CSPs: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 215
3. Intelligent Backtracking: Looking Backward: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 219
4. Complexity Calculation for CSP Problems: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 214

Constraints:

5. SudokuWiki.Org: Naked Candidates. Available at: [https://www.sudokuwiki.org/naked_candidates#:~:text=A%20Naked%20Pair%20\(also%20known,same%20row%20C%20column%20or%20box.](https://www.sudokuwiki.org/naked_candidates#:~:text=A%20Naked%20Pair%20(also%20known,same%20row%20C%20column%20or%20box.) (Accessed 16th December 2024)
6. SudokuWiki.Org: Hidden Candidates. Available at: https://www.sudokuwiki.org/Hidden_Candidates (Accessed 20th December 2024)
7. SudokuWiki.Org: Intersection Removal. Available at: https://www.sudokuwiki.org/Intersection_Removal (Accessed 26th December 2024)
8. SudokuWiki.Org: X-Wing Strategy. Available at: https://www.sudokuwiki.org/X_Wing_Strategy (Accessed 30th December 2024)
9. AC3 (Arc-Consistency) Algorithm: Norvig, P. Russel, S. (2010) Artificial Intelligence: A Modern Approach, Global Edition. Page 209

Modules:

- NumPy. Available at: <https://numpy.org/> (Accessed 6th January 2025)
- Combinations from itertools. Available at: <https://docs.python.org/3/library/itertools.html#itertools.combinations> (Accessed 6th January 2025)
- Copy. Available at: <https://docs.python.org/3/library/copy.html> (Accessed 6th January 2025)